

<https://drive.google.com/file/d/1iQcBkwro-Dj5xgd8hzhyGQr8HP4AfeJS/view>

Supervised learning + model selection

Classification (Supervised Learning)

- Classification is a **Supervised Learning** technique
- uses **labeled training data** to learn a model and **classify** new **unseen data** (unlabeled data)

Principle of Classification

- **Build** a model **using training examples**.
- **Learn characteristics** of each class.
- **Predict unknown** class labels.

Supervised vs Unsupervised Learning

 Supervised Learning	 Unsupervised Learning
Data has labels	Data has no labels
Used for Classification	Used for Clustering
> Predicts known classes > New data classified based on training set	Finds hidden groups
Example: Pass/Fail	Example: Customer Segments



Classification vs Prediction (Regression)



Classification

- Predicts **categorical values**.
- Output is a class label.
- Uses labels of training data to classify new data



Prediction

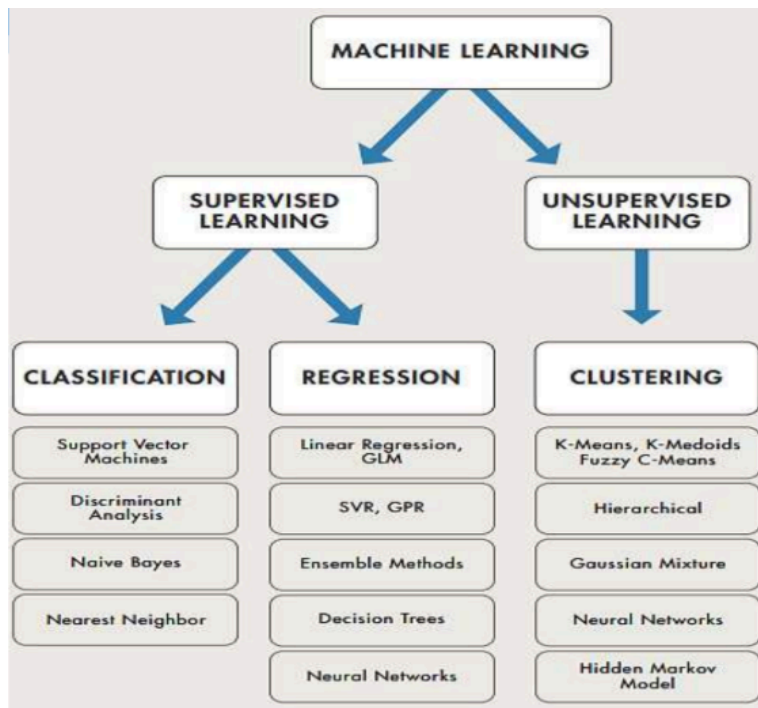
- Predicts **continuous numeric values**.
- Uses **Regression Analysis**.



Classification → Labels



Prediction → Numbers





Classification: Three-Step Process

1 Model Construction (Training)

Training Data → Classification Algorithm → Classification Model

- Use **labeled training data**.
- **Learn patterns** and rules.
- **Create** a **classifier/model**.

2 Model Testing

Test Data → Classification Model → Accuracy

- Use **separate test data**.
- **Compare predicted labels** with **actual labels**.
- Calculate **accuracy**.

Accuracy Formula

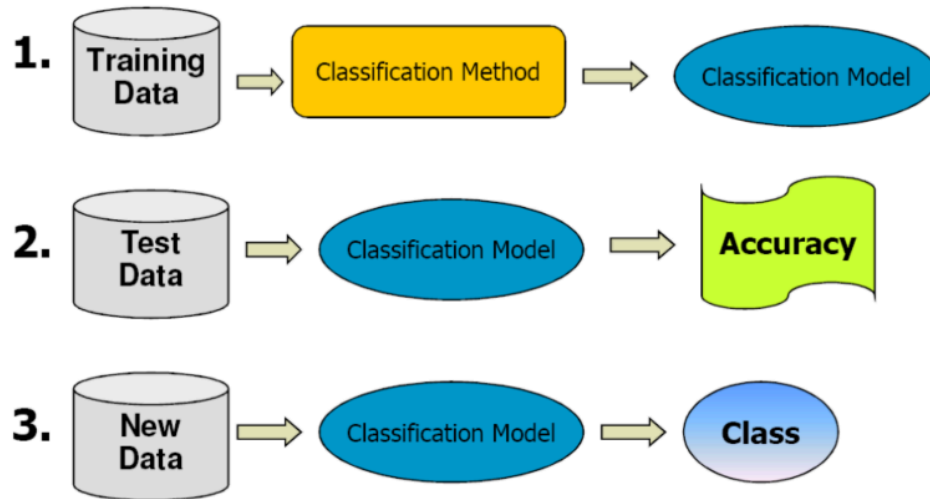
$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

⚠ Training and test data should be different to **avoid overfitting**

3 Model Usage

New Data → Classification Model → Class Label

- **Apply** the trained **model to unseen data**.
- **Predict** the correct **class**.



Model Selection

Choose **simplest model** that works

Overfitting

- **Too complex model**
- **Memorizes training data**

✓ High training accuracy

✗ Low test accuracy → **fails on test data**

Underfitting

- **Too simple model**

✗ **Poor training** accuracy

✗ **Poor test** accuracy

Data Splitting

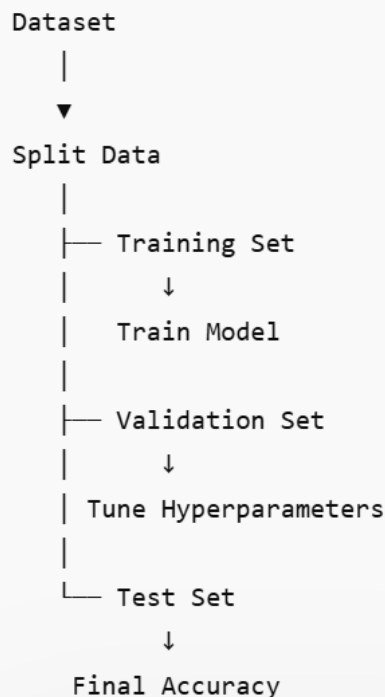
A data set is split into the following:

1. Training Set › **Train model**

2. Validation Set › **Tune parameters**

3. Test Set › **Final evaluation (use only once)**

- After training, the model is tested on validation data
- **Validation set** ⇒ only used for evaluation during development.
- If you keep checking the test set repeatedly, you'll indirectly optimize for it and get unrealistic results ⇒ this is why its only used once



K-Fold Cross Validation

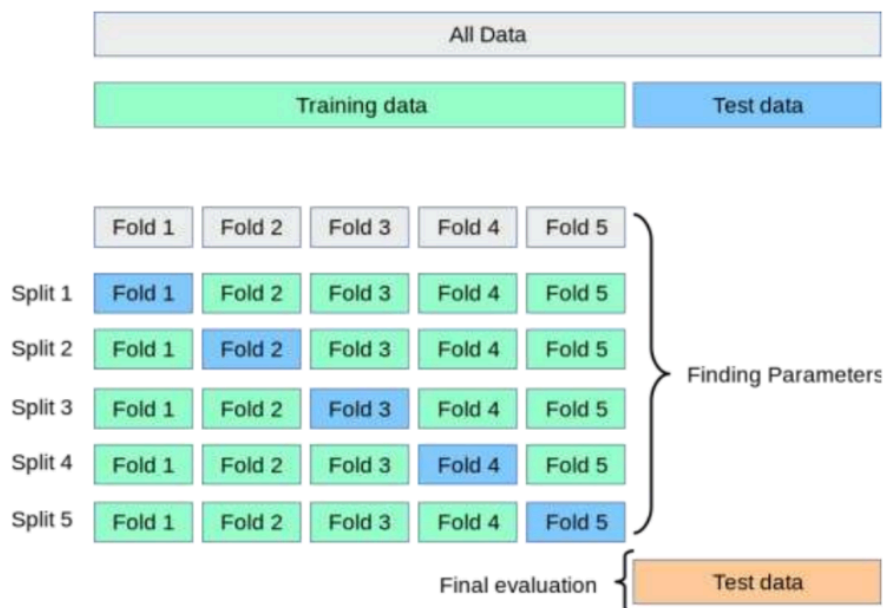
- › used to evaluate a model more reliably when **data is limited**.
- › **Each fold** gets a chance to be the **test set exactly once**.
- › **Reduces bias** due to random splitting
- › Average the errors across all k iterations for the final model performance.

Steps

1. **Split data into K parts**
2. **Train** on **K-1** folds
3. **Test** on **1 fold**
4. **Repeat K times**

Formula

$$CV = \frac{1}{K} \sum_{k=1}^K Score_k$$



Train-Test Split	K-Fold CV
One train/test split	Multiple train/test splits
Faster	Slower
Less reliable	More reliable
Data tested once	Every sample tested once

Confusion Matrix

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

Leave-One-Out Cross Validation (LOOCV)

- LOOCV is a special case of K-Fold Cross Validation where: **K = N**
- N = total number of samples.

- So instead of splitting data into 5 or 10 folds, **each individual data point becomes its own test set once.**

How It Works

For every iteration:

- **Train** on **$N - 1$ samples**
- **Test** on **1 sample**
- Repeat until every sample has been used as the test sample once

Example

Dataset = **5 samples**

A B C D E

Iteration 1

Train: B C D E

Test : A

Iteration 2

Train: A C D E

Test : B

Iteration 3

Train: A B D E

Test : C

Iteration 4

Train: A B C E

Test : D

Iteration 5

Train: A B C D

Test : E

Final Performance

Calculate the error/**accuracy** for **each iteration**.

$$\text{Final Accuracy} = \frac{\text{Sum of all accuracies}}{N}$$

Advantages

- **Uses** almost **all data for training** (**N-1 samples**)
- Good for very small datasets
- Gives a **less biased** estimate of performance

Disadvantages

- **Computationally expensive**
- Requires training the model **N times**
- Very **slow** for large datasets

Feature	K-Fold CV	LOOCV
Number of folds	K	N
Test samples per iteration	N/K	1
Training samples	K-1 folds	N-1 samples
Iterations	K	N
Speed	Faster	Slower
Training data used	Less	Maximum possible



Ockham's Razor Principle

When **multiple hypotheses/models explain the training data equally well**, choose the **simplest one**.



In Machine Learning

- If two models have **similar accuracy**:
-  **Prefer the simpler model**
-  **Avoid unnecessarily complex models**



Why Prefer Simpler Models?

1 Better Generalization

- **Performs better on unseen data.**
- **Learns actual patterns** instead of memorizing data.

2 Avoids Overfitting

- **Complex** models **may fit noise** in the training data.
- Simpler models focus on important relationships.

3 Easier to Interpret

- **Easier to understand, explain, and debug.**
- More transparent decision-making.

When several models are **consistent with the observed data**, the **simplest model should be preferred** because it generally **generalizes better, avoids overfitting, and is easier to interpret**.

Bias vs Variance

Feature	 Bias	 Variance
Meaning	Error due to overly simple assumptions	Error due to excessive sensitivity to training data
Cause	Model is too simple	Model is too complex
Model Behavior	Cannot learn the true pattern	Learns noise along with the pattern
Result	Underfitting	Overfitting
Training Error	High	Low
Testing Error	High	High
Performance on Training Data	Poor	Very good
Performance on New Data	Poor	Poor
Complexity	Low complexity	High complexity

Ideal Model

- Low Bias & Low Variance
-